설비 고장을 예측하는 AI 데이터 분석

시간 연속성 기반 전처리

측정 주기와 리샘플링 후 결측

posco

CONTENTS

목차

01 이 파트가 책임지는 것

02 시간 인덱스와 resample 복기

03 측정 주기 진단

04 리샘플링 후 결측과 asfreq

05 결측 확인과 선택 기준

posco

01

이 파트가 책임지는 것

현실 설비 데이터, 시간 축부터 바로잡기

posco

이 파트가 책임지는 것

분석 전에 시간 축부터 바로잡는 것이 이 파트의 목표

KEY POINT 01

설비 데이터는 시간 간격이 일정하지 않고 중간이 비어 있음

통신 끊김·센서 정지 같은 현장 사정 때문에, 10초 주기 설정도 실제로는 12초 뒤 또는 넘게 빈 채로 들어옴

KEY POINT 02

분석 전에 시간 축부터 바로잡는 것이 이 파트의 목표

새 라이브러리가 아니라 이미 아는 Pandas로 현실 데이터의 시간 축을 바로잡는 실전 기술

posco

학습용 데이터와 현실 데이터의 차이

가장 큰 차이는 시간 간격과 빈 시점의 존재

구분	앞 파트 (학습용)	이 파트 (현실)
시간 간격	일정함	들쭉날쭉
빈 시점	없음	단절·누락
분석 전 처리	거의 불필요	정규화 + 누락 처리

> 현실 데이터에는 **주기 정규화 + 누락 처리** — 시간 연속성 기반 전처리 필요

이 파트의 두 가지 책임

측정 주기를 일정하게 통일하고, 비었던 시점을 **합리적으로 보완**

01 · 측정 주기

들쭉날쭉한 시간 간격을 다루는 일

일정한 기준 주기로 통일

이동평균·상관관계 계산이 일정 간격을 전제로 동작하므로 주기 통일이 우선

들쭉날쭉 간격 → 동일 주기 격자

02 · 누락 처리

드러난 빈 시점을 다루는 일

앞뒤 값으로 합리적으로 채우기

정규화 과정에서 드러나거나 원래 비어 있던 시점을 적절한 방법으로 보완

빈 칸 → 신뢰할 수 있는 값

두 가지를 모두 거쳐야 신뢰할 수 있는 분석이 가능

posco

02

시간 인덱스와 resample 복기

오늘 실습에 바로 쓰는 핵심 빠른 복습

posco

시간 인덱스 만들기 3단계

글자를 시간 타입으로 → 인덱스로 → 시간순 정렬, 세 단계의 흐름

STEP 1

시간 변환

to_datetime으로 글자를 시간 타입으로 변환. 빨셈·정렬 같은 시간 계산이 가능해짐

`pd.to_datetime`

→

STEP 2

인덱스 설정

set_index로 시간 컬럼을 인덱스로 올림. 시간 기준 슬라이싱·집계의 기반

`df.set_index`

→

STEP 3

시간 정렬

sort_index로 시간순 정렬. 여러 곳에서 모은 데이터의 시각이 섞여 있어도 자동으로 줄을 세움

`df.sort_index`

이 세 단계는 어떤 시계열 데이터에서도 동일하게 적용

posco

시간 인덱스 만들기 코드

CSV를 불러와 시간 인덱스로 만드는 3단계

CODE

```
# CSV 불러오기
df = pd.read_csv("..//data//22_열처리.csv")

# timestamp 컬럼을 datetime 타입으로 변환
df["timestamp"] = pd.to_datetime(df["timestamp"])

# 인덱스로 올리고 시간순 정렬
df = df.set_index("timestamp").sort_index()
print(df.index)
```

posco

resample 빠른 복기

구간으로 묶고 + 함수로 합치기, 두 동작의 결합

01 · 구간 묶기

시간을 일정 구간으로 나누는 동작

10초 데이터를
단위로 모음

같은 분에 속한 여러 값을 하나의 묶음으로 정리

```
df.resample("1min")
```

02 · 합치는 함수

묶은 값들을 하나의 값으로

mean·max 같은
집계 함수 필수

함수가 없으면 묶기만 하고 값이 안 나옴 — 반드시 함수 호출

```
.mean() .max() .min()
```

resample은 두 동작을 한 줄에 묶어서 표현하는 도구

posco

최신 주기 표기 규칙

옛 대문자 표기 대신 소문자 s·min·소문자 h 사용

단위	❌ 옛 표기	✅ 최신 표기
초	`10S`	`10s`
분	`1T`	`1min`
시간	`1H`	`1h`

> 사소해 보여도 **에러의 흔한 원인** — 최신 표기로 통일 권장

posco

실습 1. 시간 인덱스와 1분 리샘플링 재확인

시각 글자를 시간 인덱스로 만들고 1분 평균으로 복기

목표

시각 글자를 시간 인덱스로 만들고 1분 평균으로 리샘플링을 복기

단계

· 데이터를 불러와 시각 열을 시간 인덱스로 올리고 정렬

· 두 센서를 1분 단위 평균으로 묶기

· 1분 칸 중 값이 빈 칸이 있는지 확인

예상 결과

1분 평균 표가 만들어지고, 누락 때문에 빈 칸이 16개

posco

실습 1. 시간 인덱스와 1분 리샘플링 재확인 정답

시각 글자를 시간 인덱스로 만들고 1분 평균으로 복기

CODE · 실습 1

```
import pandas as pd
df = pd.read_csv('..//data//22_열처리.csv')
df['timestamp'] = pd.to_datetime(df['timestamp'])
df = df.set_index('timestamp').sort_index()
one_min = df[['제어출력', '소입로온도']].resample('1min').mean()
print(one_min.head(3).round(2))
print('1분 평균 빈 칸:', int(one_min['제어출력'].isna().sum()))   # 16
```

03

측정 주기 진단

측정 주기의 정의 · 불일치 원인 · 간격 진단 방법

측정 주기란 무엇인가

측정 주기 = 센서가 값을 한 번 기록하고 다음 값을 기록하기까지의 시간 간격

DEFINITION

측정 주기는 센서가 값을 기록하는 시간 간격

영어로는 sampling interval — 방금 실습의 "약 10초마다"가 바로 이것

WHY IT MATTERS

이 간격이 일정해야 시계열 분석이 제대로 동작

"값 6개가 곧"이라는 계산이 성립하려면 간격이 일정해야 함

posco

측정 주기 — 짧을 때와 길 때

데이터 양과 **짧은 이상 신호** 사이의 선택

01 · 짧은 주기

1초 주기 → 60개, 1시간 3600개

짧은 변화까지
포착, 데이터 양 폭증

자세한 관찰이 가능하지만 저장·처리 부담이 커짐

디테일 ↑ · 부담 ↑

02 · 긴 주기

값이 듬성하게 들어옴

가벼운 데이터,
짧은 이상 신호 누락

저장은 가볍지만, 그 사이에 일어난 짧은 이상은 놓치기 쉬움

부담 ↓ · 디테일 ↓

측정 주기는 디테일과 부담 사이의 선택

posco

고장 예측엔 짧은 주기

10초는 설비 모니터링에서 흔히 쓰는 현실적 설정

KEY POINT 01

고장의 전조는 짧은 순간에 나타나는 경우가 많음

베어링이 깨지기 직전의 진동 급변처럼, 짧은 신호가 결정적. 주기가 길면 놓침

KEY POINT 02

우리 샘플의 10초 주기는 현실적인 모니터링 설정

고장 예측용 센서는 비교적 짧은 주기로 설정됨 — 10초가 흔한 선택

그런데 주기가 항상 일정할지 다음 슬라이드에서 확인

posco

규칙적 주기와 불규칙 주기

실습 데이터는 불규칙 주기 — 설정은 10초지만 실제는 제각각

구분	✅ 규칙적	❌ 불규칙
시간 간격	항상 동일	들쭉날쭉
분석 적용	바로 가능	정규화 후
우리 샘플	해당 없음	해당함

> 10초, 12초, 80초처럼 들쭉날쭉 — **우리가 다룰 실습 데이터**

posco

분석 도구는 규칙적 주기를 가정

불규칙 데이터는 분석 전 규칙적인 형태로 변환 필요

대부분의 시계열 분석 도구는 일정한 간격을 전제로 동작

이동평균·표준편차·자기상관 — 모두 일정 간격을 전제

6개 평균이 어떤 때는 1분, 어떤 때는 5분이 되면 결과 해석 불가

간격이 들쭉날쭉하면 "6개 평균"의 의미 자체가 불안정

불규칙한 데이터는 먼저 규칙적인 형태로 변환 필요

이게 이 파트의 큰 주제 — 시간 축을 일정하게 맞추기

posco

측정 주기가 흐트러지면

두 가지 위험 — 속도 오독과 신호 누락

위험 01 · 변화 속도 오독

같은 변화량, 다른 의미

10초 상승과
70초 상승을 구분 못 함

앞은 급격한 이상, 뒤는 완만한 변화 — 간격을 무시하면 구분 불가

급격 vs 완만 식별 불가

위험 02 · 신호 누락

결정적 순간을 통째로 놓침

이상이 나타난 순간
데이터가 통째로 빔

고장은 짧은 순간에 신호를 보냄 — 그 시점이 비어 있으면 예측 불가

예측 자체가 불가능

두 위험 모두 분석 결과의 신뢰도를 무너뜨림

측정 주기 정리는 필수 작업

"있으면 좋은" 작업이 아니라 "안 하면 고장을 놓치는" 필수 작업

KEY POINT 01

측정 주기 정리는 안 하면 고장을 놓치는 필수 작업

진동 스파이크가 마침 센서 단절 구간에 발생했다면 그 데이터는 영영 사라짐

KEY POINT 02

설비 분석에서 전처리가 분석만큼 중요한 이유

빈 구간을 합리적으로 복원하는 일이 전처리의 본질

posco

측정 주기 불일치란

"센서는 10초마다 보낸다고 했는데, 실제 기록은 그렇지 않은 상태"

DEFINITION

설정 주기와 실제 기록 간격이 어긋난 상태

설정은 10초인데 실제는 10초·13초·80초로 제각각 — 주기가 불일치

중요한 구분

정렬돼 있어도 간격은 얼마든지 불규칙할 수 있음

데이터가 시간순으로 정렬됐다는 것과 간격이 일정하다는 것은 전혀 다른 이야기

posco

불일치의 두 형태

대처 방식이 다른 미세한 흔들림과 큰 벌어짐

형태 01 · 미세한 흔들림

값은 있지만 시각이 부정확

10초가 9초·11초·12초로
조금씩 어긋남

기준 주기로 다시 정렬하면 대부분 해결되는 경증 케이스

재정렬로 해결

형태 02 · 큰 벌어짐

데이터 누락에 가까운 상태

10초가 80초로 벌어져
여러 시점이 통째로 빔

그 사이 시점이 통째로 빠진 상태 — 빠진 값을 추정해 채워야 함

값 추정으로 보완

두 형태 모두 정규화 + 결측 처리의 대상

posco

현장에서 주기가 어긋나는 이유

이상적이지 않은 현장 사정에서 비롯되는 세 가지 원인

원인	무슨 일	데이터 모습
통신 지연	전송이 밀림	10초가 13초로
센서 단절	값이 기록 안 됨	구간 전체가 빔
시스템 부하	기록 시각 흔들림	미세하게 불규칙

> 우리 샘플에도 **약 70초가 비는 구간**이 존재 — 센서 단절로 추정

posco

우리가 할 수 있는 건 후처리

흐트러짐을 탓하기보다 원인을 이해하고 보정하는 것이 분석가의 일

REALITY

현장 데이터는 항상 흐트러진 채로 들어옴

현장 설비는 완벽하지 않고, 통신과 시스템 변수도 끊임없이 영향을 줌

OUR JOB

받는 쪽에서 후처리로 바로잡는 수밖에 없음

원인을 막을 수는 없지만 결과는 보정할 수 있음 — 분석가의 역할

후처리는 데이터 분석가의 핵심 책임 영역

posco

다중 센서 주기 정렬 문제

여러 센서의 주기가 다르면 같은 시각에 모든 값이 모이지 않음

실제 설비는 진동·온도·압력 등 여러 센서를 동시에 사용

한 설비가 하나의 센서로 끝나지 않음 — 복합 모니터링이 일반적

센서마다 측정 주기가 다르면 같은 시각에 모든 값이 있지 않음

진동 10초·온도 30초·압력이면 동일 시각에 세 값이 동시에 있는 일이 드묾

같은 시각의 값이 없으면 센서 간 관계 분석 불가

"이 순간 진동과 온도의 관계는?" 같은 질문에 답을 줄 수 없음

posco

공통 시간축으로 정렬

여러 센서를 함께 분석하려면 같은 시각 기준으로 줄 세우기

SOLUTION 01

모든 센서를 같은 시간 격자에 맞춰 정렬

10초라는 공통 축을 정하고 모든 센서 값을 그 축에 올림

SOLUTION 02

결국 공통 주기로 리샘플링한 뒤 결측 처리

다중 센서 정렬도 결국 리샘플링 + 결측 처리와 같은 흐름 — 시간 축 정렬

posco

측정 간격 진단 방법

시각끼리 빼서 간격 분포를 확인하는 단순한 기술

연속된 두 시각의 차이를 구하면 간격이 일정한지 확인 가능

인접한 두 timestamp의 뺄셈이 진단의 출발점

차이가 모두 10초면 규칙적, 제각각이면 불규칙

숫자 분포 하나로 데이터의 성격이 한눈에 드러남

diff로 시각끼리 빼고, value_counts로 간격별 개수 확인

```
.diff() → .dt.total_seconds() → .value_counts()
```

posco

측정 간격 진단 코드

시각끼리 빼서 간격 분포 확인

CODE

```
# 시각끼리 빼서 초 단위 차이 구하기
sec = df.index.to_series().diff().dt.total_seconds()

# 간격별 개수를 정렬해서 확인
print(sec.value_counts().sort_index().head())

# 10.0초 169회, 최대 간격 80초
```

posco

진단 결과 읽기

가장 흔한 간격을 기준 주기로 정해 정규화

READING 01

한 간격이 압도적이고 나머지가 소수면 불일치 섞인 것

"기본 주기는 그것인데 일부 불일치가 섞임"으로 해석. 여러 숫자가 비슷하면 매우 불규칙

READING 02

가장 흔한 간격을 기준 주기로 정해 정규화

10초가 압도적이면 10초가 기준 — 불필요한 빈 칸이나 값 손실 방지

실습 2. 측정 간격 분포로 주기 진단

이웃 간 시각 차이로 정상 주기와 벌어진 구간 찾기

목표

연속된 두 측정 시각의 간격을 구해 정상 주기와 벌어진 구간을 찾기

단계

· 시간 인덱스의 이웃 간 차이를 초 단위로 구하기

· 간격 값의 분포를 세어 가장 흔한 정상 주기를 확인

· 가장 크게 벌어진 간격을 확인

예상 결과

대부분 10초(정상 주기), 가장 크게 벌어진 간격은 80초

posco

실습 2. 측정 간격 분포로 주기 진단 정답

이웃 간 시각 차이로 정상 주기와 벌어진 구간 찾기

CODE · 실습 2

```
gaps = df.index.to_series().diff().dt.total_seconds()
print(gaps.value_counts().sort_index().head())
# 10.0초가 169회로 최다, 최대 간격은 80초
print('최대 간격(초):', gaps.max())                    # 80.0
```

posco

04

리샘플링 후 결측과 asfreq

NaN의 발생 원리 · asfreq와 resample의 차이

업샘플링 시 NaN이 생기는 원리

빈 칸을 만드는 것이 아니라 원래 비어 있던 시점을 드러냄

일정한 시간 격자를 새로 만들고 기존 값을 시각이 맞는 칸에 올림

먼저 새 격자를 만들고, 그 다음 기존 값을 해당 칸에 배치

원래 빠졌던 시점은 채울 값이 없어 NaN으로 남음

"징검다리 사이에 빈 디딤판을 새로 놓는" 모양 — 새 디딤판이 NaN

이렇게 주기를 촘촘하게 만드는 것이 업샘플링

빈 칸을 만드는 게 아니라 — 원래 비어 있던 시점을 드러내는 일

posco

NaN은 에러가 아니다

NaN은 오류가 아니라 정보 — "원래 값이 없었다"는 정직한 표시

NAN의 의미

NaN은 여기에 원래 값이 없었다는 정직한 표시

임의 값으로 슬그머니 채우면 실제값과 추정값이 섞여 구분 불가

다음 주제

이 NaN을 어떻게 채울지가 이 파트의 핵심 주제

NaN으로 남겨두기 때문에 신중하게 채울 방법을 고를 수 있음

다운샘플링과 업샘플링의 결측

결측 관점에서 정반대로 동작하는 두 방향

구분	다운샘플링	업샘플링
주기 변화	길게	짧게
값 처리	묶어 합침	새 칸 생성
결측	줄어듦	드러나서 늘어남

> 다운샘플링은 값을 모아 채우고, 업샘플링은 새 칸을 만들어 **빈 칸을 드러냄**

정규화엔 두 방향이 섞인다

하나의 정규화 안에 합쳐짐과 새 NaN이 공존

대체로 10초인데 일부가 어긋난 데이터를 10초로 맞추면

"10초 정규화" 한 줄에 두 가지 변환이 함께 발생

촘촘한 구간은 다운샘플링처럼 묶이고, 빈 구간은 업샘플링처럼 NaN 생성

"그냥 10초로 맞췄을 뿐인데 왜 어떤 곳은 비지?"가 자연스러운 결과

그래서 정규화 후엔 반드시 결측 확인 필요

"정규화 → 결측 확인"의 순서가 늘 함께 가는 이유

리샘플링 결측의 특성

빈 칸의 시간 위치가 명확한 결측 — 보간의 토대

일정한 격자 위에 올렸기에 빈 칸의 시간 위치가 명확

"언제" 비어 있는지 시간 축 위에 정확히 표시됨

앞뒤 값과 시간 간격을 알면 빈 시점 값을 합리적으로 추정

09:01:00에 0.50, 09:01:20에 0.54라면 10초 시점은 약 0.52로 추정

이것이 다음 시간에서 배울 보간의 토대

앞뒤 값으로 빈 칸을 메우는 일이 곧 보간(interpolation)

posco

리샘플링 다음은 결측 처리

정규화 먼저, 채우기 나중 — 순서를 뒤집으면 위치가 불분명

ORDER 01

먼저 시간 축을 일정하게 맞추고 그다음 빈 칸을 채움

정규화하지 않은 원본은 행 자체가 없어 빠진 사실조차 알기 어려움

ORDER 02

이 순서가 이 파트 전체를 관통하는 큰 줄기

일정한 격자로 정규화하면 "여기 칸이 비었다"가 명확히 드러남

posco

asfreq — 격자에 올려놓기

값을 합치거나 계산하지 않고 있는 값을 제자리에 놓을 뿐

ASFREQ 01

정해진 주기 격자에 값을 그대로 올려놓음

.asfreq("10s") — 10초 격자를 만들고 기존 값을 시각이 맞는 칸에 배치

ASFREQ 02

값이 없는 칸은 NaN, 합치거나 계산하지 않음

원본 값 보존 — 업샘플링에 잘 맞는 도구

posco

asfreq 기본 사용

10초 격자에 그대로 올려놓기

CODE

```
# 제어출력 컬럼을 10초 격자로 정규화
result = df[["제어출력"]].asfreq("10s")

# 결과: 총 200 행, NaN 22 개
print("총 행:", len(result))
print(result["제어출력"].isna().sum())
```

posco

asfreq와 resample의 성격

한 마디로, asfreq는 원본 보존, resample은 구간 대표값

Q1 **asfreq는 어떻게 동작하나**

격자의 각 시각에 값이 있으면 가져오고 없으면 NaN — 계산은 안 함

Q2 **resample은 어떻게 동작하나**

시간 구간으로 나눠 평균 같은 함수로 값을 합침

Q3 **그래서 무엇이 다른가**

asfreq는 원본 보존, resample은 구간 대표값 계산

posco

격자 올리기 vs 구간 묶기

한 줄 차이 — 함수 필요 여부

01 · asfreq

격자 위에 그대로 놓기

격자에 그대로 올려놓기, 합치지 않음

합치지 않으니 집계 함수가 필요 없음 — 원본 보존

```
df.asfreq("10s")
```

02 · resample

구간 안의 값을 합쳐 대표값으로

구간으로 묶어 합치기, 함수 필요

mean·max 같은 함수를 반드시 붙여야 결과가 나옴

```
df.resample("10s").mean()
```

이 한 줄의 차이가 빈 칸을 다루는 방식을 결정

posco

asfreq와 resample의 결측 차이

같은 데이터에서 asfreq 22개, resample 16개 — 결측 개수가 다름

Q1 asfreq는 빈 칸을 어떻게

정확한 시각에 값 없으면 NaN, 어긋난 값은 버려짐

Q2 resample은 빈 칸을 어떻게

구간 안에 값 하나라도 있으면 평균이 채워짐

Q3 결과는

resample이 asfreq보다 빈 칸이 적게 나오는 경우 많음 — 우리 샘플도 22 vs 16

우열이 아닌 성향 차이

어느 쪽이 맞는지가 아니라 목적에 따라 선택

USE ASFREQ

원본 보존이 목적이면 asfreq

원래 측정값을 그대로 보존하고 싶을 때 적합

USE RESAMPLE

구간 대표값과 노이즈 감소가 목적이면 resample

구간 대표값으로 정리하고 노이즈를 줄이고 싶을 때 적합

posco

asfreq의 ffill과 bfill 옵션

격자를 만들면서 동시에 빈 칸을 채우는 method 옵션

01 · ffill

forward fill — 앞값을 앞으로

method='ffill'로
빈 칸을 바로 앞값으로

이전 측정값을 그대로 끌어와 채움

```
.asfreq("10s", method="ffill")
```

02 · bfill

backward fill — 뒷값을 뒤로

method='bfill'로
빈 칸을 바로 뒷값으로

이후 측정값을 앞으로 끌어와 채움 — NaN 0개

```
.asfreq("10s", method="bfill")
```

옵션 하나로 NaN을 0개로 만들 수 있음

posco

method 옵션 결과 비교

옵션 유무에 따른 NaN 차이

CODE

```
# 옵션 없이 — NaN 22개
a = df[["제어출력"]].asfreq("10s")
print(a["제어출력"].isna().sum())

# method='ffill' — NaN 0개
b = df[["제어출력"]].asfreq("10s", method="ffill")
print(b["제어출력"].isna().sum())
```

method 옵션의 한계

빠르지만 데이터 성격에 따라 부정확할 수 있음

LIMIT 01

앞뒤 값을 단순 복사 — 빠르지만 변하는 데이터엔 부정확

천천히 변하는 데이터엔 무난, 빠르게 변하는 데이터엔 부정확할 수 있음

LIMIT 02

정교하게 채우려면 다음 시간의 보간 사용

간단히 끝낼지, NaN으로 두고 보간으로 정교하게 채울지 데이터 성격에 따라 선택

method는 빠른 길, 보간은 정교한 길

asfreq가 적절한 상황

원본 보존 · 업샘플링 · 나중 결정 — 세 상황

01 · 원본 보존

실제 값을 살릴 때

센서가 보낸
실제 값 보존

resample은 평균으로 원본을 바꿈 — asfreq는 있는 값 그대로

값 하나하나가 중요할 때

02 · 업샘플링

주기를 촘촘하게 만들 때

주기를 더
촘촘하게 만들기

빈 칸이 드러나는 업샘플링에 자연스럽게 들어맞음

듬성 → 촘촘

03 · 나중 결정

채우는 방법을 따로 정하고 싶을 때

빈 칸을
NaN으로 두고 보기

어디가 비었는지 확인한 뒤 방법을 고르고 싶을 때

위치 보고 결정

asfreq vs resample 선택

상황별 정리

상황	도구
원본 값 보존	`asfreq`
구간 요약 · 노이즈 감소	`resample`
업샘플링 후 보간	`asfreq`
다운샘플링 평균	`resample`

원본 보존·업샘플링 = asfreq, 구간 요약·다운샘플링 = resample

다운샘플링엔 resample

1초 처럼 줄일 때 asfreq를 쓰면 59초 값을 모두 버림

1초를으로 줄이는 다운샘플링에 asfreq를 쓰면 안 됨

asfreq는 격자의 정확한 시각만 받음 — 나머지를 모두 버리는 결과

asfreq는 격자의 한 값만 가져오고 나머지 59초는 버림

데이터를 심하게 잃는 부적절한 선택

노이즈를 줄이려면 resample 평균이 적합

평균을 내며 잡음도 함께 완화 — 우열이 아니라 역할이 다름

posco

실습 3. asfreq로 정규화 후 NaN 확인

10초 격자에 값을 올려 빠진 시점을 빈 값으로 드러내기

목표

10초 격자에 값을 올려 놓아 빠진 시점을 빈 값으로 드러내기

단계

· 두 센서를 10초 격자로 정규화하기

· 정규화 후 총 행 수와 빈 값 개수를 세기

· 앞값 채움 옵션을 주면 빈 값이 채워짐을 확인

예상 결과

정규화 후 200행, 두 센서 각각 22칸 빈 값; 앞값 채움 시 0

posco

실습 3. asfreq로 정규화 후 NaN 확인 정답

10초 격자에 값을 올려 빠진 시점을 빈 값으로 드러내기

CODE · 실습 3

```
cols = ['제어출력', '소입로온도']
norm = df[cols].asfreq('10s')
print('총 행:', len(norm))                              # 200
print(norm.isna().sum())                                # 제어출력 22 / 소입로온도 22
filled = df[cols].asfreq('10s', method='ffill')
print('ffill 후:', filled.isna().sum().to_dict())       # 둘 다 0
```

05

결측 확인과 선택 기준

개수·비율·위치 확인 · 두 도구 비교 · 종합 실습

posco

리샘플링 후 결측 확인 흐름

개수 → 비율 → 위치, 세 단계로 점검

STEP 1

개수

isna().sum()으로 NaN 개수 세기 — 가장 기본 점검

`.isna().sum()`

STEP 2

비율

전체 대비 결측 비율로 심각성 판단 — 11%는 보간 가능

`.isna().mean()`

STEP 3

위치

산발인지 연속인지 패턴 확인 — 채우는 방법 결정

`plot · 산발 vs 연속`

50%가 넘으면 그 센서를 의심하는 신호

리샘플링 다음은 무조건 확인

결측 확인은 분석 전 안전 점검

CHECK 01

결측 확인은 분석 전 안전 점검과 같음

확인을 건너뛰고 이동평균을 계산하면 결과가 비거나 그럴듯하게 잘못 계산됨

CHECK 02

점검 결과에 따라 채우기 전략이 결정됨

개수·비율·위치 결과가 곧 다음 행동의 근거

posco

결측 위치를 그래프로 파악

"22개가 비었다"는 숫자 위에 어떻게 흩어졌는지

시간 축을 가로로, 센서 값을 선으로 그리면 빈 구간은 선이 끊김

점을 함께 찍으면 빈 구간이 더 분명히 보임

산발 결측은 앞뒤 값이 가까워 채우기 쉬움

한 칸씩 흩어진 빈 칸 — 10초 전후 값으로 안전하게 추정

연속 결측은 앞뒤 값이 멀어 채운 값의 신뢰도가 낮음

통째로 빈 구간은 앞뒤 거리가 멀어 추정 신뢰도가 떨어짐

posco

결측 위치 그래프 코드

선이 끊긴 구간이 **결측**

CODE

```
# 10초 격자로 정규화
norm = df[["제어출력"]].asfreq("10s")

# 점 + 선으로 함께 그려야 빈 구간이 보임
plt.plot(norm.index, norm["제어출력"], marker=".")
plt.ylabel("제어출력")
plt.show()
```

산발 결측과 연속 결측

패턴에 따라신뢰도가 다름

01 · 산발 결측

한 개씩 흩어진 빈 칸

앞뒤 가까워
채우기 쉬움

10초 전후 값을 알면 중간값을 꽤 정확히 추정 가능 — 안전한 경우

추정 신뢰도 높음

02 · 연속 결측

한 구간이 통째로 빔

신뢰도 낮아
주의 필요

우리 샘플의 약 70초 누락 구간 — 그 사이 변화는 확인 불가

표시해 두기

결측 패턴이 채우는 방법의 선택을 좌우

posco

resample과 asfreq 선택 기준

세 가지 축, 원본 보존 · 변환 방향 · 노이즈

01 · 원본 보존

실제 값을 살릴지 요약할지

실제 값 살리면 asfreq
요약 괜찮으면 resample

원본 그대로 vs 평균 요약

02 · 변환 방향

촘촘하게 vs 듬성하게

업샘플링은 asfreq
다운샘플링은 resample

방향만 보면 답이 정해지기도

03 · 노이즈

부드럽게 vs 짧은 이상까지

부드럽게는 resample
짧은 이상까지는 asfreq

평균 완화 vs 디테일 보존

posco

헷갈리면 촘촘/듬성으로 판단

"촘촘하게 만들려는가, 듬성하게 만들려는가" — 한 질문이면 결정

01 · 촘촘하게

주기를 더 좁게 만드는 방향

업샘플링이 목적이면
asfreq가 자연스러움

기존 값을 격자에 올리고 빈 칸은 NaN으로 남기는 동작이 적합

02 · 듬성하게

주기를 더 넓게 묶는 방향

다운샘플링이 목적이면
resample이 자연스러움

여러 값을 합쳐야 할 때는 집계 함수가 있는 resample만 가능

한 질문에 답하면 도구는 자연스럽게 결정

둘 다 해보고 비교

정답은 하나가 아님 — 데이터를 입체적으로 이해

POINT 01

정답은 하나가 아니라 목적에 따라 다름

asfreq는 디테일을 살리고, resample은 추세를 부드럽게 보여줌

POINT 02

헷갈리면 둘 다 적용해 결과 그래프를 비교

짧은 이상은 asfreq, 장기 마모 추세는 resample 평균 — 입체적 이해

두 결과를 함께 보면 데이터를 입체적으로 이해

실습 4. 리샘플링 후 결측 위치·개수 파악

결측의 개수·비율과 빈 시각 위치를 파악

목표

정규화 후 결측의 개수·비율과 빈 시각 위치를 파악

단계

· 정규화 데이터에서 결측 개수와 비율을 구하기

· 빈 값이 있는 시각을 뽑아 개수를 세기

· 빈 시각의 첫 구간을 확인

예상 결과

결측 22칸(11퍼센트), 첫 빈 시각은 9시 2분 30초

posco

실습 4. 리샘플링 후 결측 위치·개수 파악 정답

결측의 개수·비율과 빈 시각 위치를 파악

CODE · 실습 4

```
norm = df[['제어출력', '소입로온도']].asfreq('10s')
print('결측 개수:', norm.isna().sum().to_dict())        # 22, 22
print('결측 비율(%):', (norm.isna().mean() * 100).round(1).to_dict())  # 11.0, 11.0
miss = norm[norm['제어출력'].isna()].index
print('빈 시각 개수:', len(miss), '/ 첫 구간:', miss[0])  # 22 / 09:02:30
```

posco

실습 5. asfreq와 resample 결측 비교

같은 정규화라도 결측 수가 왜 다른지 비교

목표

같은 10초 정규화라도 asfreq와 resample의 결측이 왜 다른지 비교

단계

· 같은 데이터에 격자 올리기와 구간 묶기를 각각 적용

· 두 결과를 나란히 두고 빈 칸 수를 비교

· 1분 다운샘플링으로 결측이 크게 줄어듦을 확인

예상 결과

격자 올리기 22칸, 구간 묶기 16칸; 1분 다운샘플링은 1칸

posco

실습 5. asfreq와 resample 결측 비교 정답

같은 정규화라도 결측 수가 왜 다른지 비교

CODE · 실습 5

```
a = df[['제어출력']].asfreq('10s').rename(columns={'제어출력': 'asfreq'})
r = df[['제어출력']].resample('10s').mean().rename(columns={'제어출력': 'resample'})
print(a.join(r).head(8))
print('asfreq:', int(a['asfreq'].isna().sum()),
      '/ resample:', int(r['resample'].isna().sum()))   # 22 / 16
down = df[['제어출력']].resample('1min').mean()
print('1분 다운샘플링:', int(down['제어출력'].isna().sum()))   # 1
```

실습 6. 불규칙 주기 정규화 종합 리포트

측정 기간·결측·최장 연속 결측을 담은 진단 리포트

목표

측정 기간·시점 수·결측·가장 긴 연속 결측을 담은 시간축 진단 리포트를 작성

단계

· 정규화 데이터의 측정 기간과 시점 수를 출력

· 결측 개수와 비율을 정리

· 빈 값이 연속으로 이어진 가장 긴 길이를 구하기

예상 결과

200시점·결측 22칸(11퍼센트), 가장 긴 연속 결측은 10칸

실습 6. 불규칙 주기 정규화 종합 리포트 정답

측정 기간·결측·최장 연속 결측을 담은 진단 리포트

CODE · 실습 6

```
norm = df[['제어출력', '소입로온도']].asfreq('10s')
print('측정 기간:', norm.index.min(), '~', norm.index.max())
print('시점 수:', len(norm), '/ 결측:', norm.isna().sum().to_dict())
print('결측 비율(%):', (norm.isna().mean() * 100).round(1).to_dict())
isna = norm['제어출력'].isna()
groups = (isna != isna.shift()).cumsum()
print('가장 긴 연속 결측(칸):', int(isna.groupby(groups).sum().max()))   # 10
```